Performance Evaluation of Bandit Learning Algorithm

Part I: Classical Bandit Algorithm

Problem 1.1.

Given the parameters of the Bernoulli distributions from an oracle as the table:

Arm j	1	2	3
θ_j	0.7	0.5	0.4

The theoretically maximized expectation of aggregate rewards over N time slots should be:

$$E = N \cdot \arg\max_j \theta_j = 5000 \times 0.7 = 3500$$

In [1]:
```
import numpy as np
N = 5000
true_probs = [0.7, 0.5, 0.4]
id = np.argmax(true_probs)
oracle = N*true_probs[id]
print(oracle)
```

```
3500.0
```

Problem 1.2. Implemention

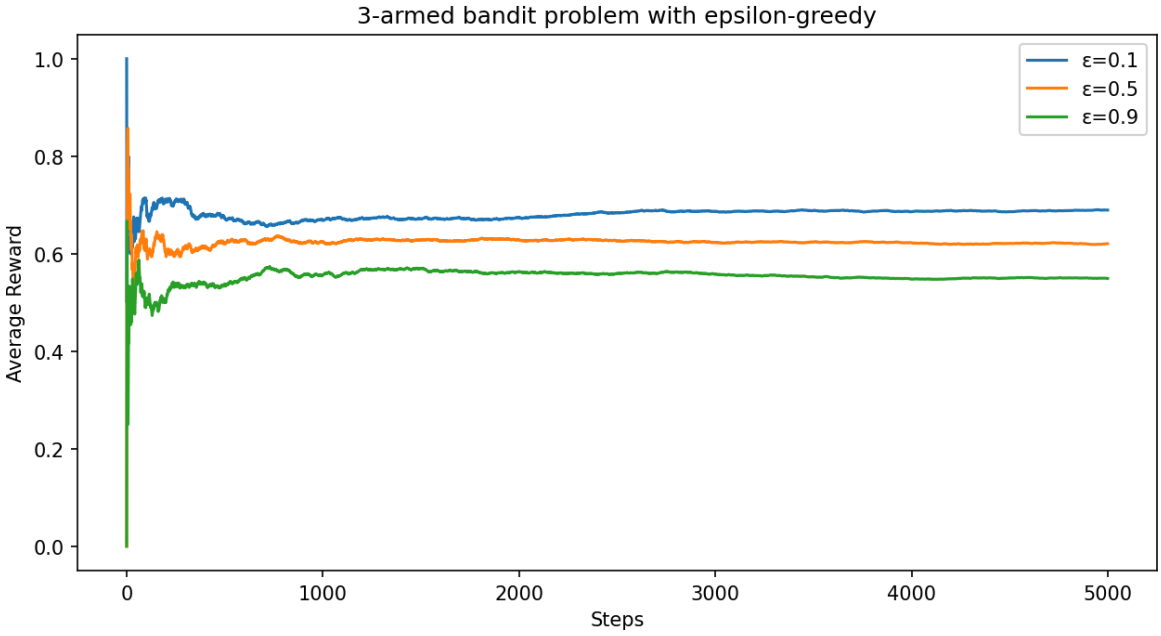
In [2]:
```
import numpy as np
import random
from scipy.stats import beta
import matplotlib.pyplot as plt


N = 5000
num_trials = 200
arms = [0, 1, 2]
true_probs = [0.7, 0.5, 0.4]

# epsilon-greedy
def epsilon_greedy(epsilon, N, true_probs):
    num_arms = len(true_probs)
    rewards = np.zeros(num_arms)
    counts = np.zeros(num_arms)
    average_reward = np.zeros(N)
    total_reward = 0

    for t in range(N):
        if random.random() < epsilon:
```

```
            arm = random.choice(range(num_arms))
        else:
            #arm = np.argmax(rewards / (counts + 1e-5))
            arm = np.argmax(rewards)
        reward = np.random.rand() < true_probs[arm]
        counts[arm] += 1
        rewards[arm] += ((reward-rewards[arm]) / counts[arm])
        total_reward += reward
        average_reward[t] = total_reward/(t+1)


    return total_reward,average_reward

# UCB
def ucb(c, N, true_probs):
    num_arms = len(true_probs)
    rewards = np.zeros(num_arms)
    counts = np.ones(num_arms)
    average_reward = np.zeros(N)
    total_reward = 0

    for arm in range(num_arms):
        counts[arm] += 1
        reward = np.random.rand() < true_probs[arm]
        rewards[arm] += reward
        total_reward += reward
        average_reward[arm] = total_reward/(arm+1)

    for t in range(num_arms, N):
        ucb_values = rewards + c * np.sqrt(2*np.log(t + 1) / counts)
        arm = np.argmax(ucb_values)
        reward = np.random.rand() < true_probs[arm]
        counts[arm] += 1
        rewards[arm] += ((reward-rewards[arm]) / counts[arm])
        total_reward += reward
        average_reward[t] = total_reward/(t+1)

    return total_reward,average_reward

# Thompson Sampling
def thompson_sampling(alpha, beta, N, true_probs):
    num_arms = len(true_probs)
    alphas = np.array(alpha)
    betas = np.array(beta)
    average_reward = np.zeros(N)
    total_reward = 0

    for t in range(N):
        sampled_theta = [np.random.beta(alphas[i], betas[i]) for i in range(num_
        arm = np.argmax(sampled_theta)
        reward = np.random.rand() < true_probs[arm]
        total_reward += reward
        average_reward[t] = total_reward/(t+1)

        alphas[arm] += reward
        betas[arm] += (1 - reward)

    return total_reward, average_reward

def run_experiments():
```

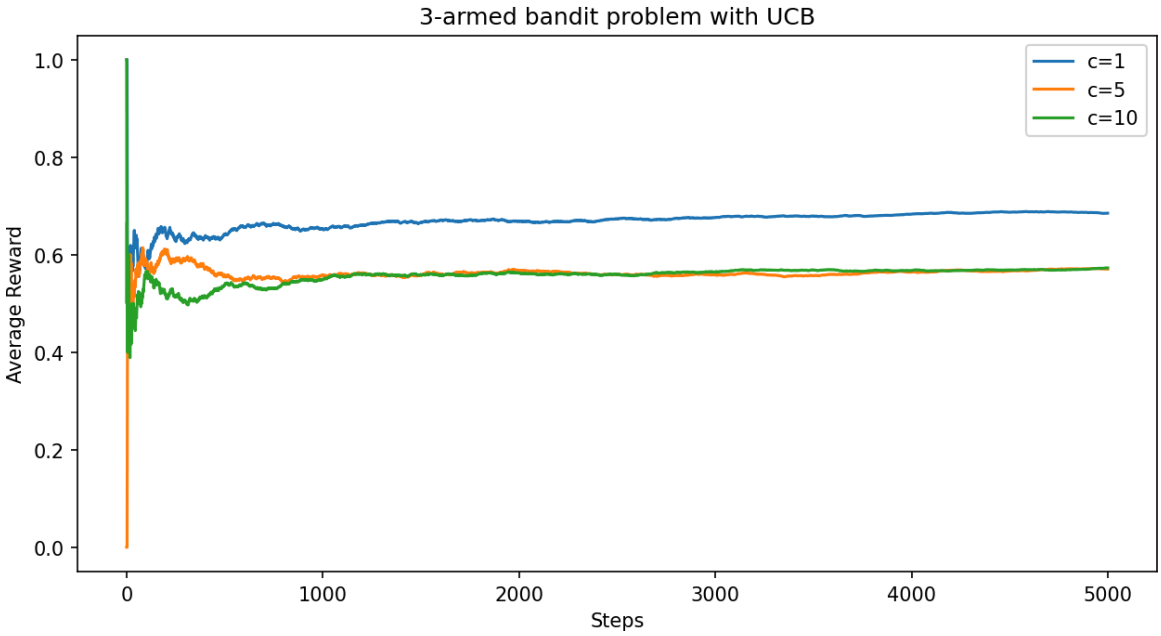
```
epsilon_params = [0.1,0.5,0.9]
ucb_params = [1, 5, 10]
ts_params = [([1, 1, 1], [1, 1, 1]), ([601, 401, 2], [401, 601, 3])]

results = {
    "epsilon_greedy": {e: [] for e in epsilon_params},
    "ucb": {c: [] for c in ucb_params},
    "thompson_sampling": {str(params): [] for params in ts_params},
}
average_rewards = {
    "epsilon_greedy": {e: [] for e in epsilon_params},
    "ucb": {c: [] for c in ucb_params},
    "thompson_sampling": {str(params): [] for params in ts_params},
}

for _ in range(num_trials):

    for e in epsilon_params:
        tot,r = epsilon_greedy(e, N, true_probs)
        results["epsilon_greedy"][e].append(tot)
        average_rewards["epsilon_greedy"][e].append(r)

    for c in ucb_params:
        tot,r = ucb(c, N, true_probs)
        results["ucb"][c].append(tot)
        average_rewards["ucb"][c].append(r)

    for params in ts_params:
        alphas, betas = params
        tot,r = thompson_sampling(alphas, betas, N, true_probs)
        results["thompson_sampling"][str(params)].append(tot)
        average_rewards["thompson_sampling"][str(params)].append(r)

print("\nAverage rewards over {} trials:".format(num_trials))
plt.figure(figsize=(10,5),dpi = 150)
for e in epsilon_params:
    res1 = np.mean(results["epsilon_greedy"][e])
    plt.plot(average_rewards["epsilon_greedy"][e][0],label=f'ε={e}')
    print(f"Epsilon-Greedy: (epsilon={e}):", res1)
    print(f"Error:", oracle-res1)
plt.legend()
plt.xlabel('Steps')
plt.ylabel('Average Reward')
plt.title('3-armed bandit problem with epsilon-greedy')
plt.show()

plt.figure(figsize=(10,5),dpi = 150)
for c in ucb_params:
    res2 = np.mean(results["ucb"][c])
    plt.plot(average_rewards["ucb"][c][0],label=f'c={c}')
    print(f"UCB (c={c}):", res2)
    print(f"Error:", oracle-res2)
plt.legend()
plt.xlabel('Steps')
plt.ylabel('Average Reward')
plt.title('3-armed bandit problem with UCB')
plt.show()

plt.figure(figsize=(10,5),dpi = 150)
for params in ts_params:
```

```
        res3 = np.mean(results["thompson_sampling"][str(params)])
        plt.plot(average_rewards["thompson_sampling"][str(params)][0],label=f'{p
        print(f"Thompson Sampling {params}:", res3)
        print(f"Error:", oracle-res3)
    plt.legend()
    plt.xlabel('Steps')
    plt.ylabel('Average Reward')
    plt.title('3-armed bandit problem with TS')
    plt.show()

if __name__ == "__main__":
    run_experiments()
```

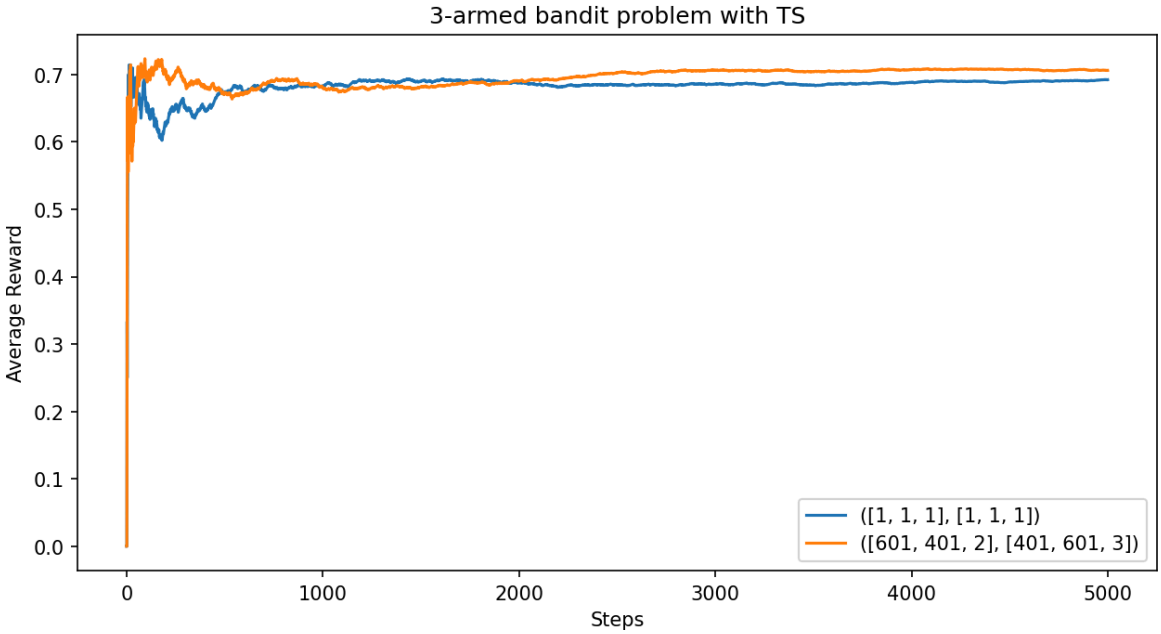
```
Average rewards over 200 trials:
Epsilon-Greedy: (epsilon=0.1): 3409.465
Error: 90.53499999999985
Epsilon-Greedy: (epsilon=0.5): 3079.73
Error: 420.27
Epsilon-Greedy: (epsilon=0.9): 2747.755
Error: 752.2449999999999
```




```
UCB (c=1): 3411.28
Error: 88.7199999999998
UCB (c=5): 2977.44
Error: 522.56
UCB (c=10): 2829.495
Error: 670.5050000000001
```




```
Thompson Sampling ([1, 1, 1], [1, 1, 1]): 3481.485
Error: 18.514999999999873
Thompson Sampling ([601, 401, 2], [401, 601, 3]): 3487.445
Error: 12.554999999999836
```



Problem 1.4: Result Analysis

For the epsilon-greedy algorithm, when $\epsilon = 0.1$ the error is around 90 and is the minimum. As the epsilon increase to 0.5 and 0.9, the error increase to around 410 and 750. So the smaller the epsilon is, the better the estimator performs.

For the UCB algorithm, when $c = 1$ the error is around 90 is the smallest. As c increase to 5 and 10, the error increase to around 520 and 670. So the smaller the c is, the better the estimator performs.

For the Thompson Sampling algorithm, when the initial beta distriution is: $(\alpha_1, \beta_1) = (601, 401), (\alpha_2, \beta_2) = (401, 601), (\alpha_3, \beta_3) = (2, 3)$, the error is around 8 and is the smallest. When the initial beta distribution is uniform, the

error is around 17. Actually the closer the initial beta distribution is to the ground truth, the better the estimator performs. However, we don't know what the groud truth is, so we may need to choose uniform beta distribution.

Among all the three algorithm, the best is the Thompson Sampling algorithm.

Problem 1.5: Understanding of the "Exploration-exploitation Trade-off"

The exploration means trying different options to gather more information about their rewards. And the exploitation means using current knowledge to select the option that appears best.

This problem needs the exploration-exploitation trade-off for the reasons below:

- **The information is limited:** The true distribution of θ is uncertain at the beginning. And we should learn it from the data and experience. Thus we need exploration. If we have too much exploitation, we may get stuck in local optima and miss the potentially better options.
- **Optimization:** The final goal of the question is to maximize our total rewards, purely random exploration may be impossible to get the best answer. Although we can run many times exploration to estimate the distribution of θ. However, it cost a lot. Therefore we need exploitation. If we have too much exploration, we may wastes opportunities on suboptimal arms. The speed of convergence to optimal strategy can be very slow.

Then we look deep into each algorithm to see how different algorithm handle it.

- **Epsilon-Greedy:** Uses ϵ probability for exploration, $(1-\epsilon)$ for exploitation. it is simple and effective because we can directly control it through ϵ. However, the exploration is uniformly random, which doesn't learn from past explorations. That will cause waste.
- **UCB:** It combines exploitation (mean reward) with exploration bonus. It will give options with a higher reward or fewer attempts a higher probability to choose. And exploration term decreases with more observations. Therefore it can automatically balances exploration and exploitation based on uncertainty
- **Thompson Sampling:** It uses posterior probability matching and update it after each trial. It can also automatically adapts exploration based on uncertainty level

Problem 1.6: Dependent Case

- **Definition:** The dependent case means that the probability and conditional probability is nat the same. When we pull different arms, the probability is not

simply multiply the success probabilities of each arms. The correlation of different arms $Cov(X_i, X_j) \neq 0$ for some $i, j \in \{1, 2, \ldots\ldots\}$. s.t.

$$P(X) \neq P(X|Y)$$

- **Instance:** However, our problem need N times pulling, which is complicated to decide the $Cov(X_{i_1}, X_{i_2}, \ldots\ldots, X_{i_N})$. We choose a simple case that only depend on the last pulling:

  There are two arms X, Y, with two success probability $\theta_1 = 0.5, \theta_2 = 0.7$. For each of them, $X \sim Bern(\theta_1), Y \sim Bern(\theta_2)$. Denote X_i means the i-th pull is the X arm.

  Then we introduce denpendence to it: when the last pulling and the present pulling is not the same arm, the probability changes as below:

  $$P(X_{i+1} = 1|Y_i) = 0.6$$

  $$P(X_{i+1} = 0|Y_i) = 0.4$$

  $$P(Y_{i+1} = 1|X_i) = 0.9$$

  $$P(Y_{i+1} = 0|X_i) = 0.1$$

  However, $P_{X_{i+1}|X_i}, P_{Y_{i+1}|Y_i}$ do not change.

- **Optimal:** For $N = 2000$, the optimal is first choose arm X and then choose X, Y alternately. Which has the maximum reward about $1000 \times (0.6 + 0.9) = 1500$

- **Checking:** Use the three algorithms above to check the accuray:

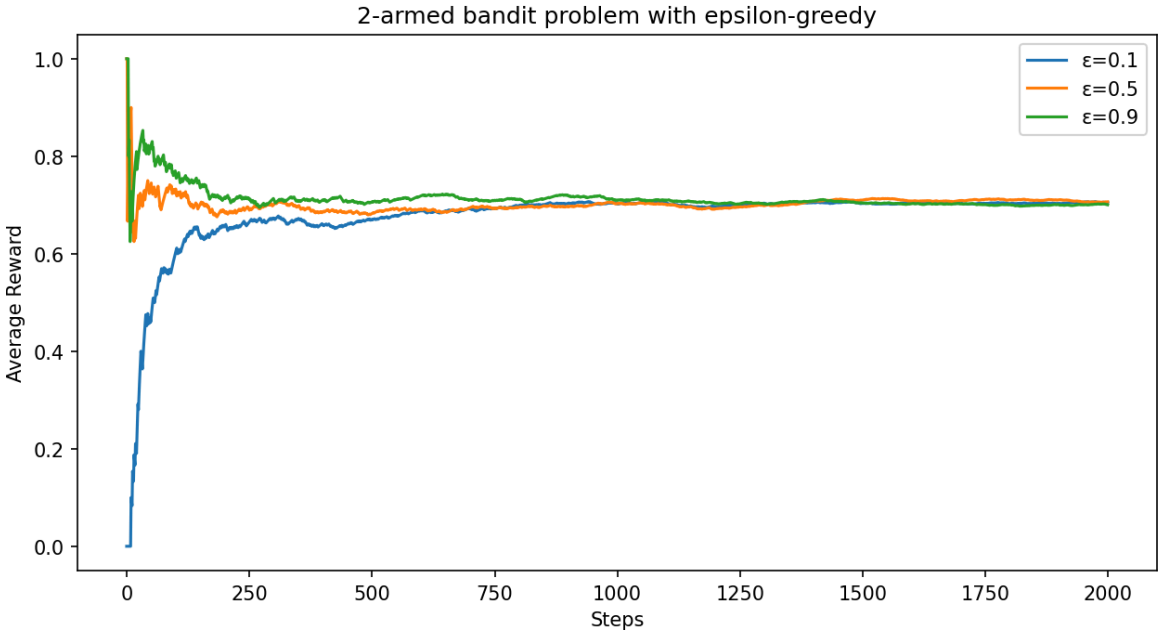
In [83]:
```python
import numpy as np
N = 2000
true_probs = [0.5, 0.7]
condition_probs = [0.6, 0.9]
oracle = 1500

# epsilon-greedy
def epsilon_greedy(epsilon, N, true_probs):
    num_arms = len(true_probs)
    rewards = np.zeros(num_arms)
    counts = np.zeros(num_arms)
    average_reward = np.zeros(N)
    total_reward = 0
    last_arm = 0

    for t in range(N):
        if random.random() < epsilon:
            arm = random.choice(range(num_arms))
        else:
            arm = np.argmax(rewards)
        if last_arm == 0 or last_arm == arm + 1:
            reward = np.random.rand() < true_probs[arm]
        elif last_arm == 1:
            reward = np.random.rand() < condition_probs[arm]
```

```
        elif last_arm == 2:
            reward = np.random.rand() < condition_probs[arm]
        counts[arm] += 1
        rewards[arm] += ((reward-rewards[arm]) / counts[arm])
        total_reward += reward
        average_reward[t] = total_reward/(t+1)
        last_arm = arm+1

    return total_reward,average_reward

# UCB
def ucb(c, N, true_probs):
    num_arms = len(true_probs)
    rewards = np.zeros(num_arms)
    counts = np.ones(num_arms)
    average_reward = np.zeros(N)
    total_reward = 0
    last_arm = 0

    for arm in range(num_arms):
        counts[arm] += 1
        if last_arm == 0 or last_arm == arm + 1:
            reward = np.random.rand() < true_probs[arm]
        elif last_arm == 1:
            reward = np.random.rand() < condition_probs[arm]
        elif last_arm == 2:
            reward = np.random.rand() < condition_probs[arm]
        rewards[arm] += reward
        total_reward += reward
        average_reward[arm] = total_reward / (arm + 1)
        last_arm = arm + 1

    for t in range(num_arms, N):
        ucb_values = rewards + c * np.sqrt(2*np.log(t + 1) / counts)
        arm = np.argmax(ucb_values)
        if last_arm == 0 or last_arm == arm + 1:
            reward = np.random.rand() < true_probs[arm]
        elif last_arm == 1:
            reward = np.random.rand() < condition_probs[arm]
        elif last_arm == 2:
            reward = np.random.rand() < condition_probs[arm]
        counts[arm] += 1
        rewards[arm] += ((reward-rewards[arm]) / counts[arm])
        total_reward += reward
        average_reward[t] = total_reward / (t + 1)
        last_arm = arm + 1

    return total_reward,average_reward

# Thompson Sampling
def thompson_sampling(alpha, beta, N, true_probs):
    num_arms = len(true_probs)
    alphas = np.array(alpha)
    betas = np.array(beta)
    average_reward = np.zeros(N)
    total_reward = 0
    last_arm = 0

    for t in range(N):
        sampled_theta = [np.random.beta(alphas[i], betas[i]) for i in range(num_
```

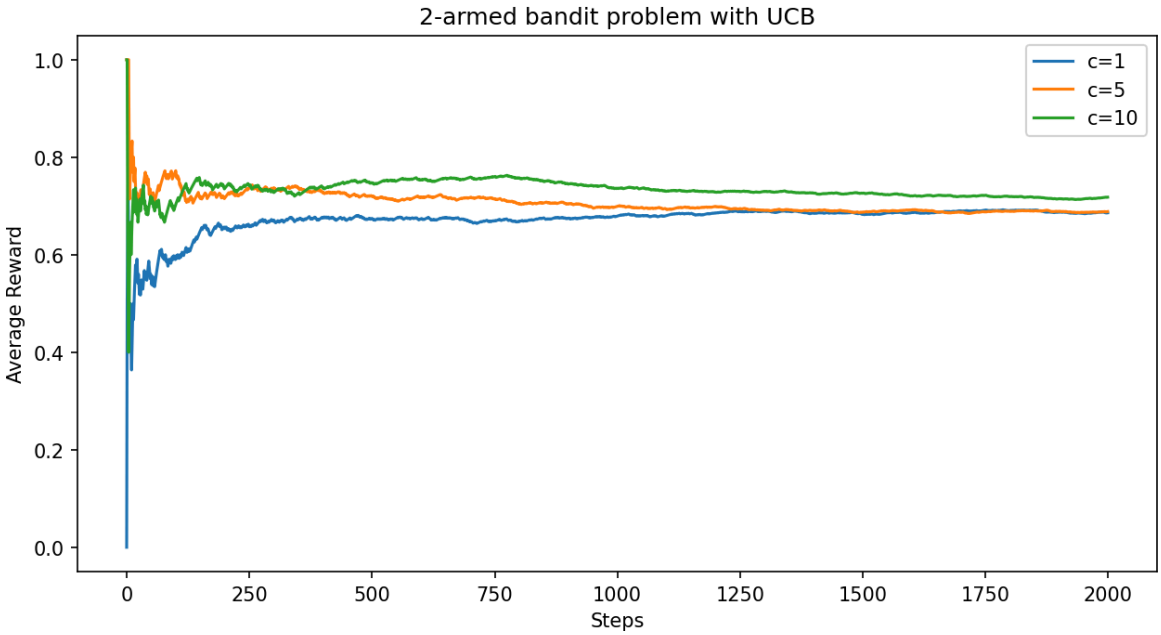
```
        arm = np.argmax(sampled_theta)
        if last_arm == 0 or last_arm == arm + 1:
            reward = np.random.rand() < true_probs[arm]
        elif last_arm == 1:
            reward = np.random.rand() < condition_probs[arm]
        elif last_arm == 2:
            reward = np.random.rand() < condition_probs[arm]
        total_reward += reward
        average_reward[t] = total_reward/(t+1)

        alphas[arm] += reward
        betas[arm] += (1 - reward)
        last_arm = arm + 1

    return total_reward, average_reward

def run_experiments():
    epsilon_params = [0.1,0.5,0.9]
    ucb_params = [1, 5, 10]
    ts_params = [([1, 1], [1, 1]), ([401,601], [601,401])]

    results = {
        "epsilon_greedy": {e: [] for e in epsilon_params},
        "ucb": {c: [] for c in ucb_params},
        "thompson_sampling": {str(params): [] for params in ts_params},
    }
    total_rewards = {
        "epsilon_greedy": {e: [] for e in epsilon_params},
        "ucb": {c: [] for c in ucb_params},
        "thompson_sampling": {str(params): [] for params in ts_params},
    }

    for _ in range(num_trials):
        for e in epsilon_params:
            tot,r = epsilon_greedy(e, N, true_probs)
            results["epsilon_greedy"][e].append(tot)
            total_rewards["epsilon_greedy"][e].append(r)

        for c in ucb_params:
            tot,r = ucb(c, N, true_probs)
            results["ucb"][c].append(tot)
            total_rewards["ucb"][c].append(r)

        for params in ts_params:
            alphas, betas = params
            tot,r = thompson_sampling(alphas, betas, N, true_probs)
            results["thompson_sampling"][str(params)].append(tot)
            total_rewards["thompson_sampling"][str(params)].append(r)

    print("\nAverage rewards over {} trials:".format(num_trials))
    plt.figure(figsize=(10,5),dpi = 150)
    for e in epsilon_params:
        res1 = np.mean(results["epsilon_greedy"][e])
        plt.plot(total_rewards["epsilon_greedy"][e][0],label=f'ε={e}')
        print(f"Epsilon-Greedy: (epsilon={e}):", res1)
        print(f"Error:", oracle-res1)
    plt.legend()
    plt.xlabel('Steps')
    plt.ylabel('Average Reward')
    plt.title('2-armed bandit problem with epsilon-greedy')
```

```
    plt.show()

    plt.figure(figsize=(10,5),dpi = 150)
    for c in ucb_params:
        res2 = np.mean(results["ucb"][c])
        plt.plot(total_rewards["ucb"][c][0],label=f'c={c}')
        print(f"UCB (c={c}):", res2)
        print(f"Error:", oracle-res2)
    plt.legend()
    plt.xlabel('Steps')
    plt.ylabel('Average Reward')
    plt.title('2-armed bandit problem with UCB')
    plt.show()

    plt.figure(figsize=(10,5),dpi = 150)
    for params in ts_params:
        res3 = np.mean(results["thompson_sampling"][str(params)])
        plt.plot(total_rewards["thompson_sampling"][str(params)][0],label=f'{par
        print(f"Thompson Sampling {params}:", res3)
        print(f"Error:", oracle-res3)
    plt.legend()
    plt.xlabel('Steps')
    plt.ylabel('Average Reward')
    plt.title('2-armed bandit problem with TS')
    plt.show()

if __name__ == "__main__":
    run_experiments()
```

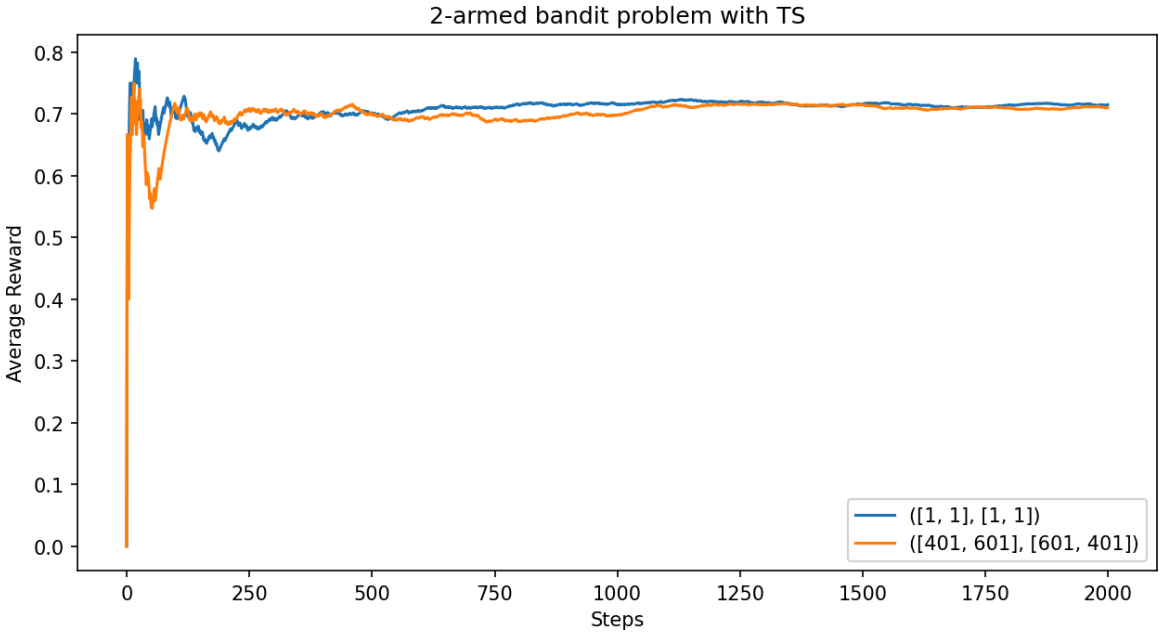
```
Average rewards over 200 trials:
Epsilon-Greedy: (epsilon=0.1): 1402.05
Error: 97.95000000000005
Epsilon-Greedy: (epsilon=0.5): 1409.61
Error: 90.3900000000001
Epsilon-Greedy: (epsilon=0.9): 1368.145
Error: 131.85500000000002
```



```
UCB (c=1): 1380.94
Error: 119.05999999999995
UCB (c=5): 1392.06
Error: 107.94000000000005
UCB (c=10): 1451.085
Error: 48.914999999999964
```




```
Thompson Sampling ([1, 1], [1, 1]): 1404.06
Error: 95.94000000000005
Thompson Sampling ([401, 601], [601, 401]): 1397.86
Error: 102.1400000000001
```



We found the original algorithm perform not very good. Because all the algorithms' exploitation is to select the maximum θ that estimated using past data. However that will lose the information of alternative choice. So it is more important to exploration in this question. That is also why in UCB algorithm, when we increase c the error declined. It has more pobability to exploration. However it is still not good enough.

- **Design Algorithm:** We can modify Thompson sampling. Such as taking conditionanl probability into consideration, we set $X_{i+1}|Y_i \sim Beta(1,1)$, $Y_{i+1}|X_i \sim Beta(1,1)$ and then update them. Every time we use DP to determine which arm to choose (at the same time we should also do more exploration).

$$OPT(X,i) = \max\{X|Y + OPT(Y,i-1), X|X + OPT(X,i-1)\}$$

$$OPT(Y,i) = \max\{Y|Y + OPT(Y,i-1), Y|X + OPT(X,i-1)\}$$

$$solution = \max\{OPT(X,N), OPT(Y,N)\}$$

Everytime we update 4 probabilities $P_{X|X}, P_{Y|X}, P_{X|Y}, P_{Y|Y}$.

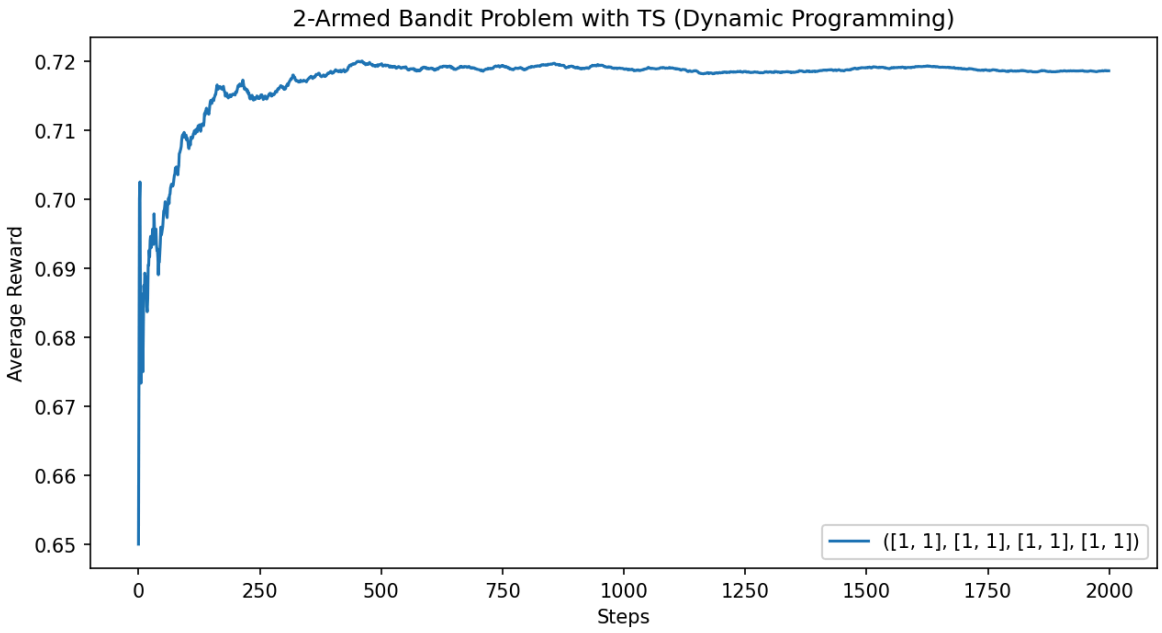
In [4]:
```
def greedy(N,x_x,x_y,y_x,y_y):
    OPTx = [0] * (N + 1)
    OPTy = [0] * (N + 1)

    OPTx[1] = x_x
    OPTy[1] = y_y

    for i in range(1, N + 1):

        OPTx[i] = max(x_y + OPTy[i-1], x_x + OPTx[i-1])
        OPTy[i] = max(y_y + OPTy[i-1], y_x + OPTx[i-1])
    RES =[OPTx[N], OPTy[N]]
    return np.argmax(RES)


def thompson_sampling(alpha, c_alpha, beta, c_beta, N, true_probs, condition_pro
    num_arms = len(true_probs)
    alphas = np.array(alpha)
    betas = np.array(beta)
    c_alphas = np.array(c_alpha)
    c_betas = np.array(c_beta)
    average_reward = np.zeros(N)
    total_reward = 0
    last_arm = 0
    flag = 0

    for t in range(N):
        sampled_theta = [np.random.beta(alphas[i], betas[i]) for i in range(num_
        sampled_lamda = [np.random.beta(c_alphas[i], c_betas[i]) for i in range(
        arm = greedy(t+1,sampled_theta[0],sampled_lamda[0],sampled_lamda[1],samp

        if flag==3:
            flag = 0
            arm = 1-(last_arm-1)
        #print(arm)
        if last_arm == 0 or last_arm == arm + 1:
            flag+=1
            reward = np.random.rand() < true_probs[arm]
        elif last_arm == 1:
            reward = np.random.rand() < condition_probs[arm]
            c_alphas[arm] += reward
            c_betas[arm] += (1 - reward)
        elif last_arm == 2:
            reward = np.random.rand() < condition_probs[arm]
```

```
            c_alphas[arm] += reward
            c_betas[arm] += (1 - reward)
        total_reward += reward
        average_reward[t] = total_reward/(t+1)

        alphas[arm] += reward
        betas[arm] += (1 - reward)
        last_arm = arm + 1

    return total_reward, average_reward
```

In [5]:

```
import numpy as np
import matplotlib.pyplot as plt

N = 2000
true_probs = [0.5, 0.7]
condition_probs = [0.6, 0.9]
oracle = 1500


def run_experiments():
    num_trials = 100
    ts_params = [([1, 1], [1, 1], [1, 1], [1, 1])]
    results = {
        "thompson_sampling": {str(params): [] for params in ts_params},
    }
    total_rewards = {
        "thompson_sampling": {str(params): [] for params in ts_params},
    }

    for _ in range(num_trials):
        for params in ts_params:
            alphas, betas, c_alphas, c_betas = params

            tot, avg_reward = thompson_sampling(alphas, c_alphas, betas, c_betas


            results["thompson_sampling"][str(params)].append(tot)
            total_rewards["thompson_sampling"][str(params)].append(avg_reward)


    plt.figure(figsize=(10, 5), dpi=150)
    for params in ts_params:
        res3 = np.mean(results["thompson_sampling"][str(params)])
        plt.plot(np.mean(total_rewards["thompson_sampling"][str(params)], axis=0
        print(f"Thompson Sampling {params}: Average Total Reward = {res3}")
        print(f"Error: Oracle = {oracle}, Observed = {res3}, Error = {oracle - r

    plt.legend()
    plt.xlabel('Steps')
    plt.ylabel('Average Reward')
    plt.title('2-Armed Bandit Problem with TS (Dynamic Programming)')
    plt.show()

if __name__ == "__main__":
    run_experiments()
```

```
Thompson Sampling ([1, 1], [1, 1], [1, 1], [1, 1]): Average Total Reward = 1437.2
3
Error: Oracle = 1500, Observed = 1437.23, Error = 62.76999999999998
```



- **Result Analysis:** In this problem, the optimal solution is strongly dependent on the exploration. However in this problem, UCB algorithm can already perform very well when we pick higher c, because we need more weighted on exploration. And the instance is a simple two-state bandit, for more arms we can construct a Markov Chain to model the problem with q_{ij} means the conditional probability when pulling i arm first and j arm next.

Part II: Bayesian Bandit Algorithms

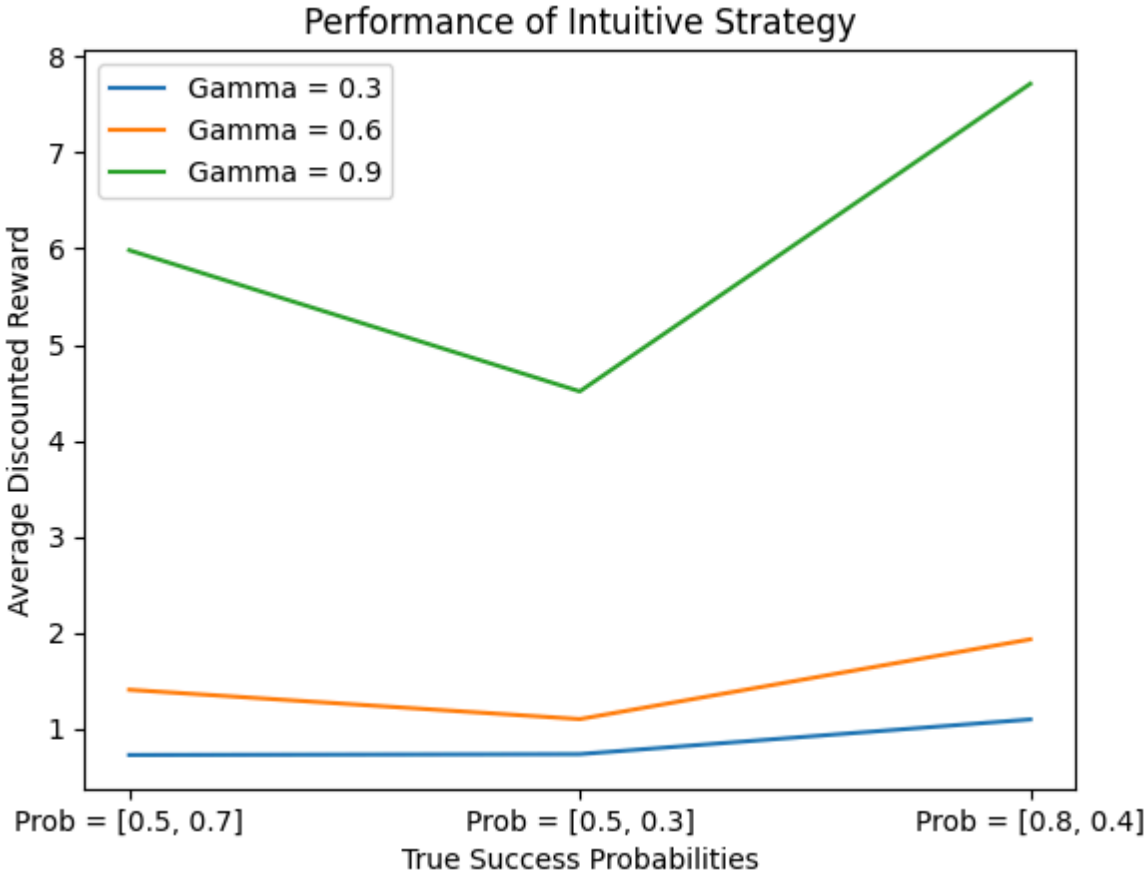
Problem 2.1: Intuitive policy

In [85]:
```python
import numpy as np
import random
import tqdm
import matplotlib.pyplot as plt


def initialize_bandit(num_arms):
    alpha = [1] * num_arms
    beta = [1] * num_arms
    return alpha, beta

# select
def select_arm(alpha, beta):
    expected_values = [a / (a + b) for a, b in zip(alpha, beta)]
    return np.argmax(expected_values)

# update
def update_posterior(alpha, beta, chosen_arm, success):
    if success:
        alpha[chosen_arm] += 1
    else:
        beta[chosen_arm] += 1
```

```python
def run_experiment(num_arms, num_rounds, true_probs, gamma):
    alpha, beta = initialize_bandit(num_arms)
    total_reward = 0
    gamma_pow = 1

    for t in range(1, num_rounds + 1):

        chosen_arm = select_arm(alpha, beta)

        success = random.random() < true_probs[chosen_arm]

        reward = gamma_pow if success else 0
        total_reward += reward

        update_posterior(alpha, beta, chosen_arm, success)

        gamma_pow *= gamma

    return total_reward


def run_multiple_experiments(num_experiments, num_arms, num_rounds, true_probs_l
    results = []
    for gamma in gamma_list:
        avg_rewards = []
        for true_probs in true_probs_list:
            total_rewards = []
            for _ in tqdm.tqdm(range(num_experiments), desc=f"Gamma = {gamma}"):
                reward = run_experiment(num_arms, num_rounds, true_probs, gamma)
                total_rewards.append(reward)
            avg_rewards.append(np.mean(total_rewards))
        results.append((gamma, avg_rewards))
    return results


def visualize_results(results, true_probs_list):
    for gamma, avg_rewards in results:
        plt.plot(range(len(true_probs_list)), avg_rewards, label=f"Gamma = {gamm
    plt.xticks(range(len(true_probs_list)), [f"Prob = {probs}" for probs in true
    plt.xlabel("True Success Probabilities")
    plt.ylabel("Average Discounted Reward")
    plt.title("Performance of Intuitive Strategy")
    plt.legend()
    plt.show()


def print_average_rewards(results):
    for gamma, avg_rewards in results:
        print(f"Results for Gamma = {gamma}:")
        for i, avg_reward in enumerate(avg_rewards):
            print(f"  True Probabilities {i + 1}: Average Reward = {avg_reward:.
        print("\n" + "-" * 40)


if __name__ == "__main__":

    num_arms = 2
    num_rounds = 200
    num_experiments = 100
    gamma_list = [0.3, 0.6, 0.9]
```

```
    true_probs_list = [[0.5, 0.7], [0.5, 0.3], [0.8, 0.4]]


    results = run_multiple_experiments(num_experiments, num_arms, num_rounds, tr

    print_average_rewards(results)

    visualize_results(results, true_probs_list)
```

```
Gamma = 0.3: 100%|██████████| 100/100 [00:00<00:00, 600.54it/s]
Gamma = 0.3: 100%|██████████| 100/100 [00:00<00:00, 578.46it/s]
Gamma = 0.3: 100%|██████████| 100/100 [00:00<00:00, 586.12it/s]
Gamma = 0.6: 100%|██████████| 100/100 [00:00<00:00, 607.33it/s]
Gamma = 0.6: 100%|██████████| 100/100 [00:00<00:00, 516.80it/s]
Gamma = 0.6: 100%|██████████| 100/100 [00:00<00:00, 564.64it/s]
Gamma = 0.9: 100%|██████████| 100/100 [00:00<00:00, 497.06it/s]
Gamma = 0.9: 100%|██████████| 100/100 [00:00<00:00, 404.42it/s]
Gamma = 0.9: 100%|██████████| 100/100 [00:00<00:00, 429.40it/s]
Results for Gamma = 0.3:
  True Probabilities 1: Average Reward = 0.7258
  True Probabilities 2: Average Reward = 0.7348
  True Probabilities 3: Average Reward = 1.0963

----------------------------------------
Results for Gamma = 0.6:
  True Probabilities 1: Average Reward = 1.4053
  True Probabilities 2: Average Reward = 1.0997
  True Probabilities 3: Average Reward = 1.9308

----------------------------------------
Results for Gamma = 0.9:
  True Probabilities 1: Average Reward = 5.9835
  True Probabilities 2: Average Reward = 4.5133
  True Probabilities 3: Average Reward = 7.7178

----------------------------------------
```



Problem 2.2: Intuitive policy is not optimal

Suppose we have two arms bandit with success probabilities $p_1 = 0.9, p_2 = 0.1$. The stratergy may have problems below:

1. Initial Parameter Bias

If the initial parameters are given as follows:

- **Bandit 1**:
  $(\alpha_1, \beta_1) = (10, 90)$
- **Bandit 2**:
  $(\alpha_2, \beta_2) = (90, 10)$

These settings result in:

1. The **incorrect bandit** (Bandit 2) having a high initial expected value.
2. The **correct bandit** (Bandit 1) having a low initial expected value.
3. The strategy persistently selecting Bandit 2 for a long time, due to the biased initial parameters.

---

2. Discount Factor

The discount factor is set as:
$\gamma = 0.6$

This leads to:

1. The reward weight in later rounds decaying rapidly.
2. Even if the strategy eventually corrects its initial error, the cumulative reward remains extremely low.

To show it more clearly, we consider the strategy used in Thompson Sampling algorithm, it use the sampled value instead of the expectation of θ. Then we can construct the algorithm below:

In [118…

```python
def run_experiment_with_bias(num_arms, num_rounds, true_probs, gamma, initial_al

    if initial_alpha is None or initial_beta is None:
        alpha, beta = initialize_bandit(num_arms)
    else:
        alpha, beta = initial_alpha[:], initial_beta[:]

    total_reward = 0
    gamma_pow = 1

    for t in range(1, num_rounds + 1):

        chosen_arm = select_arm(alpha, beta)

        success = random.random() < true_probs[chosen_arm]

        reward = gamma_pow if success else 0
        total_reward += reward

        update_posterior(alpha, beta, chosen_arm, success)

        gamma_pow *= gamma

    return total_reward, alpha, beta


def run_random_experiment(num_arms, num_rounds, true_probs, gamma):
    alpha, beta = initialize_bandit(num_arms)
    total_reward = 0
    gamma_pow = 1

    for t in range(1, num_rounds + 1):

        chosen_arm = random.choice(range(num_arms))

        success = random.random() < true_probs[chosen_arm]

        reward = gamma_pow if success else 0
        total_reward += reward

        update_posterior(alpha, beta, chosen_arm, success)

        gamma_pow *= gamma

    return total_reward, alpha, beta
```

```python
def compare_strategies(num_arms, num_rounds, true_probs, gamma, initial_alpha, i
    print(f"Ground truth: {true_probs}")
    print(f"Initial beta distribution (alpha, beta): {list(zip(initial_alpha, in

    intuitive_reward, _, _ = run_experiment_with_bias(num_arms, num_rounds, true

    random_reward, _, _ = run_random_experiment(num_arms, num_rounds, true_probs

    print(f"Intuitive policy total reward (Expectation): {intuitive_reward}")
    print(f"Sample policy total reward: {random_reward}")

if __name__ == "__main__":
    num_arms = 2
    num_rounds = 1000
    gamma = 0.6

    true_probs = [0.9, 0.1]

    initial_alpha = [10, 90]
    initial_beta = [90, 10]

    compare_strategies(num_arms, num_rounds, true_probs, gamma, initial_alpha, i
```

```
Ground truth: [0.9, 0.1]
Initial beta distribution (alpha, beta): [(10, 90), (90, 10)]
Intuitive policy total reward (Expectation): 0.01009372985041424
Sample policy total reward: 1.4632668669726887
```

From the output above we can found that the intuitive stratergy failed.

Problem 2.3

1. Total Expected Reward for Pulling Arm 1

If arm 1 is pulled, the reward depends on whether the pull is a success or a failure:

Case 1: Success

- Success occurs with probability θ_1.
- The immediate reward is 1.
- After success, the parameters of the Beta posterior distribution are updated to $(\alpha_1 + 1, \beta_1)$.
- Future rewards are discounted by γ and depend on the updated parameters, contributing: $\gamma R(\alpha_1 + 1, \beta_1, \alpha_2, \beta_2)$.

Case 2: Failure

- Failure occurs with probability $1 - \theta_1$.
- The immediate reward is 0.
- The Beta posterior distribution is updated to $(\alpha_1, \beta_1 + 1)$.
- Future rewards are discounted by γ and depend on the updated parameters, contributing: $\gamma R(\alpha_1, \beta_1 + 1, \alpha_2, \beta_2)$.

Total Expected Reward for Arm 1

Combining the two cases, the total expected reward for pulling arm 1 is:
$R_1(\alpha_1, \beta_1) = \mathbb{E}[\theta_1] \cdot (1 + \gamma R(\alpha_1 + 1, \beta_1, \alpha_2, \beta_2)) + \mathbb{E}[1 - \theta_1] \cdot \gamma R(\alpha_1, \beta_1 + 1, \alpha_2$

From Bayesian inference, we know: $\mathbb{E}[\theta_1] = \frac{\alpha_1}{\alpha_1+\beta_1}, \quad \mathbb{E}[1 - \theta_1] = \frac{\beta_1}{\alpha_1+\beta_1}$.

Substituting these values into the equation:
$R_1(\alpha_1, \beta_1) = \frac{\alpha_1}{\alpha_1+\beta_1} \cdot (1 + \gamma R(\alpha_1 + 1, \beta_1, \alpha_2, \beta_2)) + \frac{\beta_1}{\alpha_1+\beta_1} \cdot \gamma R(\alpha_1, \beta_1 + 1, \alpha_2, \beta_2$

---

2. Total Expected Reward for Pulling Arm 2

For arm 2, we take a slightly different approach to the analysis by focusing first on the posterior updates.

Posterior Updates

- If arm 2 is pulled, the posterior distribution of θ_2 is updated as follows:
  - After success, the parameters become $(\alpha_2 + 1, \beta_2)$.
  - After failure, the parameters become $(\alpha_2, \beta_2 + 1)$.

Reward Analysis

1. **Success** occurs with probability θ_2.
   The immediate reward is 1, and future rewards (discounted by γ) depend on the updated parameters: $1 + \gamma R(\alpha_1, \beta_1, \alpha_2 + 1, \beta_2)$.

2. **Failure** occurs with probability $1 - \theta_2$.
   The immediate reward is 0, and future rewards depend on the updated parameters: $\gamma R(\alpha_1, \beta_1, \alpha_2, \beta_2 + 1)$.

Total Expected Reward for Arm 2

The total expected reward for pulling arm 2 is:
$R_2(\alpha_2, \beta_2) = \mathbb{E}[\theta_2] \cdot (1 + \gamma R(\alpha_1, \beta_1, \alpha_2 + 1, \beta_2)) + \mathbb{E}[1 - \theta_2] \cdot \gamma R(\alpha_1, \beta_1, \alpha_2, \beta_2$

Substituting the Bayesian expectations: $\mathbb{E}[\theta_2] = \frac{\alpha_2}{\alpha_2+\beta_2}, \quad \mathbb{E}[1 - \theta_2] = \frac{\beta_2}{\alpha_2+\beta_2}$, we get:
$R_2(\alpha_2, \beta_2) = \frac{\alpha_2}{\alpha_2+\beta_2} \cdot (1 + \gamma R(\alpha_1, \beta_1, \alpha_2 + 1, \beta_2)) + \frac{\beta_2}{\alpha_2+\beta_2} \cdot \gamma R(\alpha_1, \beta_1, \alpha_2, \beta_2 + 1$

---

3. Optimal Strategy

To maximize the total expected reward, the optimal strategy at any step is to pull the arm that provides the higher expected reward:
$R(\alpha_1, \beta_1, \alpha_2, \beta_2) = \max\{R_1(\alpha_1, \beta_1), R_2(\alpha_2, \beta_2)\}$.

Thus, we have derived the recursive relations:
$R_1(\alpha_1, \beta_1) = \frac{\alpha_1}{\alpha_1+\beta_1} \cdot (1 + \gamma R(\alpha_1 + 1, \beta_1, \alpha_2, \beta_2)) + \frac{\beta_1}{\alpha_1+\beta_1} \cdot \gamma R(\alpha_1, \beta_1 + 1, \alpha_2, \beta_2$

$$R_2(\alpha_2, \beta_2) = \frac{\alpha_2}{\alpha_2+\beta_2} \cdot (1 + \gamma R(\alpha_1, \beta_1, \alpha_2 + 1, \beta_2)) + \frac{\beta_2}{\alpha_2+\beta_2} \cdot \gamma R(\alpha_1, \beta_1, \alpha_2, \beta_2 + 1$$

Finally: $R(\alpha_1, \beta_1, \alpha_2, \beta_2) = \max\{R_1(\alpha_1, \beta_1), R_2(\alpha_2, \beta_2)\}$.

Problem 2.4:

Firstly, this problem is a Dynamic Programing problem. However we can not solve the equations exactly because the base case of the DP is the R value at $+\infty$. So the DP has infinity transition states and has no boundry.

However, we can solve it approximately. We know that the R value will converge to a stationary distribution at infinity. So we can run an experiment of several steps like we do in problem 2.1 to get a pair of paramaters of beta distributions $(\alpha_1, \beta_1), (\alpha_2, \beta_2)$ as the esitimated value. Then, run DP to achieve the optimal strategy!

Problem 2.5: Optimal Stratergy

From the prblems above, we found the best stratergy is to using Dynamic Programming. Here wereference to Thompson Sampling algorithm first, that is every time sample θ from each beta distruibution and select the maximum one. And the update the posterior distrubution.

In the two algorithm below, we both run experiment of 100 steps to get the estimated convergency value.

In [ ]:
```python
import numpy as np
import scipy.stats as stats

# Dictionary for memoization (cache previously calculated states)
dict_cache = {}

def calculate_mean(alpha, beta):
    return alpha / (alpha + beta)

def R(alpha1, beta1, alpha2, beta2, gamma, count, exploration_limit):
    # Check if the state is already calculated (memoization)
    if (alpha1, beta1, alpha2, beta2) in dict_cache:
        return dict_cache[(alpha1, beta1, alpha2, beta2)]

    # Exploration
    if count < exploration_limit:
        # Sample success probabilities from the posterior Beta distributions
        sampled_theta1 = np.random.beta(alpha1, beta1)
        sampled_theta2 = np.random.beta(alpha2, beta2)

        # Choose the arm with the higher sampled value
        if sampled_theta1 > sampled_theta2:
            # Pull arm 1
            mean1 = calculate_mean(alpha1, beta1)
```

```
            R1 = (mean1 * (1 + gamma * R(alpha1 + 1, beta1, alpha2, beta2, gamma
                  (1 - mean1) * gamma * R(alpha1, beta1 + 1, alpha2, beta2, gamm
            reward = R1
        else:
            # Pull arm 2
            mean2 = calculate_mean(alpha2, beta2)
            R2 = (mean2 * (1 + gamma * R(alpha1, beta1, alpha2 + 1, beta2, gamma
                  (1 - mean2) * gamma * R(alpha1, beta1, alpha2, beta2 + 1, gamm
            reward = R2

    # Exploitation
    else:
        mean1 = calculate_mean(alpha1, beta1)
        mean2 = calculate_mean(alpha2, beta2)
        reward = max(mean1, mean2)

    # Store the result in the cache and return it
    dict_cache[(alpha1, beta1, alpha2, beta2)] = reward
    return reward


if __name__ == "__main__":
    alpha1, beta1 = 1, 1
    alpha2, beta2 = 1, 1
    exploration_limit = 100
    gamma_values = [0.3, 0.6, 0.9]

    # Loop through each gamma value and calculate the reward
    for gamma in gamma_values:
        # Reset cache for each gamma
        dict_cache = {}
        # Start with initial state
        reward = R(alpha1, beta1, alpha2, beta2, gamma, count=0, exploration_lim
        print(f"Initial state: gamma = {gamma}, reward = {reward:.4f}")
```

```
Initial state: gamma = 0.3, reward = 0.7410
Initial state: gamma = 0.6, reward = 1.2950
Initial state: gamma = 0.9, reward = 5.7394
```

However, we found the result is not good enough, so we consider change sampling into expectation. That is every time calculate the mean of each beta distribution and select the maximum one and update the posterior distruibution.

In [114…

```
# Dictionary for memoization (cache previously calculated states)
dict_cache = {}

def calculate_mean(alpha, beta):
    return alpha / (alpha + beta)

def R(alpha1, beta1, alpha2, beta2, gamma, count, exploration_limit=50):

    if (alpha1, beta1, alpha2, beta2) in dict_cache:
        return dict_cache[(alpha1, beta1, alpha2, beta2)]

    # Stage 2: Switch to exploitation when enough pulls are made
    if count >= exploration_limit:
        mean1 = calculate_mean(alpha1, beta1)
        mean2 = calculate_mean(alpha2, beta2)
        return max(mean1, mean2)
```

```
    # Stage 1: Exploration (recursive calculation for R1 and R2)
    mean1 = calculate_mean(alpha1, beta1)
    R1 = (mean1 * (1 + gamma * R(alpha1 + 1, beta1, alpha2, beta2, gamma, count
          (1 - mean1) * gamma * R(alpha1, beta1 + 1, alpha2, beta2, gamma, count

    mean2 = calculate_mean(alpha2, beta2)
    R2 = (mean2 * (1 + gamma * R(alpha1, beta1, alpha2 + 1, beta2, gamma, count
          (1 - mean2) * gamma * R(alpha1, beta1, alpha2, beta2 + 1, gamma, count

    # Select the maximum reward between R1 and R2
    reward = max(R1, R2)

    # Store the result in the cache and return it
    dict_cache[(alpha1, beta1, alpha2, beta2)] = reward
    return reward



if __name__ == "__main__":
    alpha1, beta1 = 1, 1
    alpha2, beta2 = 1, 1
    exploration_limit = 50  # Threshold
    gamma_values = [0.3, 0.6, 0.9]  # Different discount factors

    # Loop through each gamma value and calculate the reward
    for gamma in gamma_values:
        # Reset cache for each gamma
        dict_cache = {}
        # Start with initial state
        reward = R(alpha1, beta1, alpha2, beta2, gamma, count=0, exploration_lim
        print(f"Initial state: gamma = {gamma}, reward = {reward:.4f}")
```

```
Initial state: gamma = 0.3, reward = 0.7511
Initial state: gamma = 0.6, reward = 1.3925
Initial state: gamma = 0.9, reward = 6.0661
```

Comparing the above two, we found the second algorithm using mean value performs better than the first one using sampled value.

Boundry Case:

We want to check if the optimal straterdy would not fail when it come across the case we discussed in problem 2.2.

In [115…

```
if __name__ == "__main__":
    alpha1, beta1 = 10, 90
    alpha2, beta2 = 90, 10
    exploration_limit = 50  # Threshold
    gamma_values = [0.3, 0.6, 0.9]  # Different discount factors

    # Loop through each gamma value and calculate the reward
    for gamma in gamma_values:
        # Reset cache for each gamma
        dict_cache = {}
        # Start with initial state
```

```
        reward = R(alpha1, beta1, alpha2, beta2, gamma, count=0, exploration_lim
        print(f"Initial state: gamma = {gamma}, reward = {reward:.4f}")
```

```
Initial state: gamma = 0.3, reward = 1.2857
Initial state: gamma = 0.6, reward = 2.2500
Initial state: gamma = 0.9, reward = 8.9583
```

Part III: Additional Thoughts

3.1 Softmax Method

In the ϵ-greedy method and the UCB method, the defect is that the choice probability of action can not be well optimized. Therefore, for action selection, our ideal situation is that the action selection probability of high yield is high, and the action selection probability of low yield or negative yield is low. To be more specific, we replace action selection probabilities with weights, and actions with high returns are assigned higher weights, and vice versa. It is easy to see that the softmax method is a good choice. The softmax method is defined as follows:

Softmax is a familiar and useful method to convert the output of a neural network into probabilities in deep learning. It is defined as:

$$\text{softmax}(x_i) = \frac{e^{x_i}}{\sum_{j=1}^{n} e^{x_j}}$$

where x_i is the output of the neural network for class i and n is the number of classes.The output of the softmax function is a vector of probabilities that sum to 1.

Here, we can use the softmax method to convert the action values into probabilities. And in order to control the randomness of the action selection, we introduce a parameter called the temperature. The action selection probability is defined as:

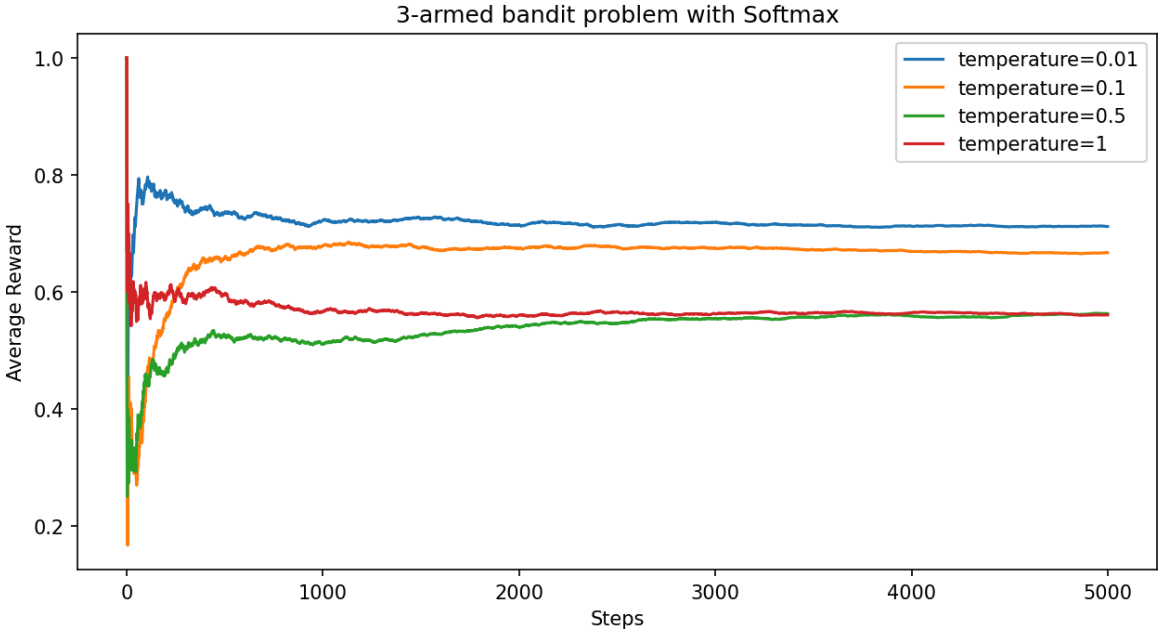
$$\pi(a) = \frac{e^{Q(a)/\tau}}{\sum_{b=1}^{n} e^{Q(b)/\tau}}$$

where $Q(a)$ is the action value of action a, and τ is a parameter called the temperature. When τ is large, the action selection is close to random, and when τ is small, the action selection is close to greedy.

In [42]:
```
def softmax(temperature, N, true_probs):
    num_arms = len(true_probs)
    rewards = np.zeros(num_arms)
    counts = np.zeros(num_arms)
    average_reward = np.zeros(N)
    total_reward = 0

    for t in range(N):
        p = np.exp(rewards / temperature) / np.sum(np.exp(rewards / temperature)
```

```
        arm = np.random.choice(range(num_arms), p=p)
        reward = np.random.rand() < true_probs[arm]
        counts[arm] += 1
        rewards[arm] += ((reward-rewards[arm]) / counts[arm])
        total_reward += reward
        average_reward[t] = total_reward/(t+1)

    return total_reward, average_reward



N = 5000
num_trials = 200
arms = [0, 1, 2]
true_probs = [0.7, 0.5, 0.4]
id = np.argmax(true_probs)
oracle = N*true_probs[id]
plt.figure(figsize=(10,5),dpi = 150)
# Softmax
temperature = [0.01,0.1,0.5,1]
for t in temperature:
    tot,r = softmax(t, N, true_probs)
    print(f"Softmax (temperature={t}):", tot)
    print(f"Error:", oracle-tot)
    plt.plot(r,label=f'temperature={t}')
plt.legend()
plt.xlabel('Steps')
plt.ylabel('Average Reward')
plt.title('3-armed bandit problem with Softmax')
plt.show()
```

```
Softmax (temperature=0.01): 3558
Error: -58.0
Softmax (temperature=0.1): 3334
Error: 166.0
Softmax (temperature=0.5): 2812
Error: 688.0
Softmax (temperature=1): 2802
Error: 698.0
```



3.2 Gradient Bandit

All the methods we have discussed so far are based on the value of the action. These methods are exactly excellent methods. However, here we introduce a new method called the gradient bandit method. As opposed to action-value methods that estimate the value of actions, gradient bandit methods don't care about the value of an action but rather learn a preference for each action over another. This preference has no interpretation in terms of reward. only the relative preference over other actions matters. The action selection probability is defined according to a softmax distribution:

$$\pi(a) = \frac{e^{H(a)}}{\sum_{i=1}^{n} e^{H(i)}}$$

where $H(a)$ is the preference of action a.

The value of $H_t(a)$ represents the preference of action a over other actions at time t. The update rule of $H_t(a)$ is defined as:

$$H_{t+1}(a) = H_t(a) + \alpha(R_t - \bar{R}_t)(1 - \pi_t(a))$$

Futhermore, it is equivalent to the following:

$$\begin{aligned} H_{t+1}(A_t) &= H_t(A_t) + \alpha\left(R_t - \bar{R}_t\right)\left(1 - \pi_t(A_t)\right), && \text{and} \\ H_{t+1}(a) &= H_t(a) - \alpha\left(R_t - \bar{R}_t\right)\pi_t(a), && \text{for all } a \neq A_t \end{aligned}$$

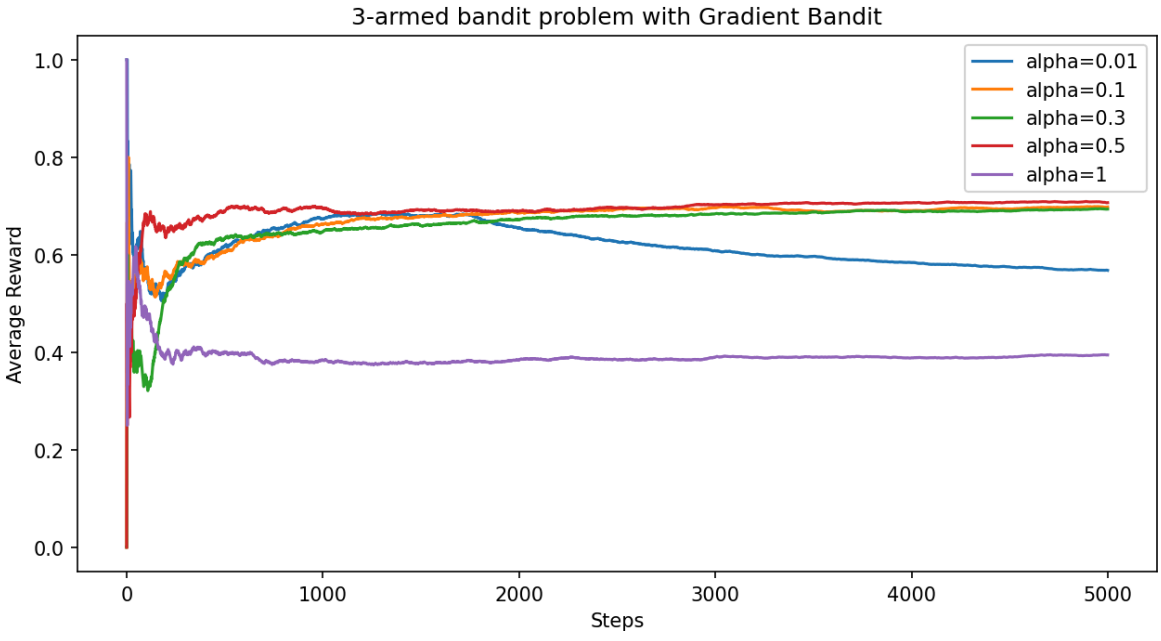
where α is the step size which determines the degree to which the preferences are changed at each step, R_t is the reward of action a at time t, $\bar{R}_t$ is the average reward at time t, and $\pi_t(a)$ is the action selection probability of action a at time t.

The inspiration of the gradient bandit is **Gradient-Descent**, a method that is widely used in optimization problems. After each update, the preference of the action is updated in the direction of the reward. If the reward is higher than the average reward, the preference of the action is increased, and vice versa.

In [82]:
```python
def GradientBandit(alpha, N, true_probs):
    num_arms = len(true_probs)
    H = np.zeros(num_arms) # preference
    rewards = np.zeros(num_arms)
    average_reward = np.zeros(N)
    total_reward = 0

    for t in range(N):
        p = np.exp(H) / np.sum(np.exp(H))
        arm = np.random.choice(range(num_arms), p=p)
        reward = np.random.rand() < true_probs[arm]
        total_reward += reward
        average_reward[t] = total_reward/(t+1)

        for a in range(num_arms):
            if a == arm:
```

```
                H[a] += alpha * (reward - np.mean(rewards)) * (1 - p[a])
            else:
                H[a] -= alpha * (reward - np.mean(rewards)) * p[a]
        rewards[arm] += reward

    return total_reward, average_reward

N = 5000
num_trials = 200
arms = [0, 1, 2]
true_probs = [0.7, 0.5, 0.4]
id = np.argmax(true_probs)
oracle = N*true_probs[id]

plt.figure(figsize=(10,5),dpi = 150)
# Gradient Bandit
alpha = [0.01,0.1,0.3,0.5,1]
for a in alpha:
    tot,r = GradientBandit(a, N, true_probs)
    print(f"Gradient Bandit (alpha={a}):", tot)
    print(f"Error:", oracle-tot)
    plt.plot(r,label=f'alpha={a}')
plt.legend()
plt.xlabel('Steps')
plt.ylabel('Average Reward')
plt.title('3-armed bandit problem with Gradient Bandit')
plt.show()
```

```
Gradient Bandit (alpha=0.01): 2840
Error: 660.0
Gradient Bandit (alpha=0.1): 3490
Error: 10.0
Gradient Bandit (alpha=0.3): 3468
Error: 32.0
Gradient Bandit (alpha=0.5): 3534
Error: -34.0
Gradient Bandit (alpha=1): 1973
Error: 1527.0
```



Here we set up different α (also called learning rate in deep learning) to control the learning speed. When α is large, the preference of the action is updated

quickly, and when α is small, the preference of the action is updated slowly. And obviously in the graph, the larger α will lead to a quicker convergence.But if α is too large, the preference of the action will be updated too quickly, which will lead to a large fluctuation in the reward.If α is too small, the preference of the action will be updated too slowly, which will lead to a slow convergence.

Divison of labor among members:

All the team members did an equal amount of work.

Name	Student Number
He Junlin	2023533113
Li Yuetong	2023533126
Li Yaxuan	2023533150